NEURAL NETWORKS WITH RELEVANCE-SORTED NODES

Luís Wolffrom Barbosa
Faculdade de Engenharia
Universidade do Porto
luiswbarbosa@gmail.com

Alípio Jorge
Faculdade de Ciências
Universidade do Porto
INESC TEC
amjorge@fc.up.pt

ABSTRACT

We present a method for training fully connected feedforward neural networks in which nodes are ranked by significance within each layer through weight manipulation. This ranking enables a structured pruning strategy that removes the least significant nodes first. We evaluate our approach against Random Pruning, Post-train Magnitude Pruning, and Group Lasso Pruning on MNIST, Fashion-MNIST, CIFAR-10, Covertype, and SDD. Our method performs competitively with other techniques, showing an advantage at high compression levels, while maintaining similarly low computational overhead.

***K*eywords** Neural Networks · Pruning · Relevance-Sorted Nodes

1 Introduction

1.1 Pruning Neural Networks

The size of neural networks has been growing exponentially since the 1950s and the growth rate has increased since 2010 [1]. The massive scale of current models has sparked interest in pruning techniques, methods for removing weights from a neural network while minimizing the impact on its predictive performance. Neural network pruning serves multiple purposes: it reduces the size of the model and, consequently, its memory and energy footprint [2], and it has also been reported to improve generalization [3]. Pruning can be classified as either unstructured, where individual weights are removed irregularly, or structured, where entire neurons or filters are removed [2]. In both cases, it is standard practice to include a brief retraining phase to allow the network to adjust its parameters [4].

Unstructured pruning typically does not result in performance gains on general-purpose hardware, as it does not reduce the size of the network. In order to obtain performance improvements, it requires specialized hardware that can exploit sparsity [2]. Structured pruning, in contrast, removes entire computational units, such as neurons or filters, physically changing the architecture of the network [5]. This structural reduction directly translates into savings in memory, computation, and energy [2].

1.2 Multilayer Perceptrons

We study multilayer perceptrons [6] with $L + 1$ layers: an input layer of index 0, an output layer, and $L - 1$ hidden layers. Each layer (except the input one) computes:

$$z^i = \sigma(W^i z^{i-1} + b^i) \tag{1}$$

Where z^i is the output of the i-th layer, W^i is the weight matrix between the $i - 1$-th and the i-th layer, b^i is the bias vector of the i-th layer and σ is the activation function. The network trains its weights and parameters to minimize a loss function $\mathcal{L}$. If θ is the set of learnable parameters, N is the number of samples in each batch, x_i is the i-th input in the batch, y_i is its target and l is the loss function for a single sample, this is expressed as

$$\min_{\theta} \mathcal{L}(\theta) = E(\theta) = \frac{1}{N} \sum_{j=1}^{N} \ell\left(z^n(x_j), y_j\right) \tag{2}$$

1.3 Regularization

Network regularization is used to force a neural network into a specific structure. This is achieved through the addition of a term to the loss function that depends on the parameters of the model rather than on its output.

$$\mathcal{L}(\theta) = E(\theta) + R(\theta) \tag{3}$$

Regularization is often used to reduce the complexity of the model. One simple regularization function, l_2 regularization, adds to the loss a component that increases with the magnitude of the model's weights. This ensures that only the important weights have high magnitudes, encouraging simpler structures.

$$R_{l_2}(\theta) = \sum_{w \in \theta} |w|^2 \tag{4}$$

1.4 Post-train Magnitude Pruning

A common assumption in the pruning literature is that the higher magnitude weights are the weights that contribute more significantly to the output of the network [2]. Accordingly, a simple approach to pruning involves ranking weights by magnitude and removing those below a certain threshold. In the case of structured pruning for multilayer perceptrons, this principle extends to entire neurons: after training a neural network, neurons are ranked by the l_2 norm of their weights, and those with the lowest norms are pruned first. This is what we call Post-train Magnitude Pruning. The l_2 norm of a vector w of weights is calculated as follows:

$$\|w\|_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2} \tag{5}$$

1.5 Group Lasso Pruning

Some structured pruning techniques also use regularization. One of them, proposed in [7] and in [8], uses Group Lasso regularization during training to promote structured sparsity. The goal of Group Lasso is to create groups of weights associated to units that can be structurally pruned (for instance, a neuron). This term encourages whole groups of weights to be zeroed out, which works better for structured pruning and can be used with Post-train Magnitude Pruning to produce pruned networks. An l_2 regularization term is also added in [8] to encourage elimination of unused weights in valuable groups, as well as the l_2 norm of the bias vector.

In Group Lasso Pruning, groups of weights can be defined as groups of incoming or outgoing weights to a neuron. The authors of [8] use both and conclude that there is no major difference between both. Our results use outgoing weight groups. Mathematically,

$$R_{GLasso}(\theta) = \alpha \sum_{l=2}^{L} \sum_{i=1}^{n_{l-1}} \|W_{:,i}^l\|_2 + \beta \left(\sum_{w \in \theta} \frac{1}{2} |w|^2 + \sum_{b \in \theta} \frac{1}{2} \|b\|^2 \right) \tag{6}$$

Where α and β are hyperparameters, n_l is the number of neurons in the l-th layer and b represents a bias vector.

2 Relevance-Sorted Network Pruning

The idea we introduce, Relevance-Sorted Network Pruning, induces a gradual decrease in weights across the neurons of a layer, thereby implicitly ranking nodes by importance: the first neurons are considered to be the most significant neurons and the last ones are the least significant neurons. This ranking easily translates into pruning by removing the least significant neurons first.

We introduce a regularization term that penalizes the incoming and outgoing weights of each node. This penalization is stronger for nodes further down the layer and results in lower weights and smaller influence on the network's output.

The regularization for each node is proportional to a penalty vector that is defined for each layer as $p[i] = i^{\alpha}$, where i is the index of the node within the layer and α is a hyperparameter that controls the rate of increase of the penalty, such that $0 < \alpha < 1$. The regularization term, considering the normalized matrix of absolute weights W and a hyperparameter β, is defined as

$$R_{RSL}(\theta) = \frac{\beta}{L-1} \sum_{l=1}^{L-1} \sum_{i=1}^{n_l} p_l[i] \left(|W_{i,:}^{l}| + |W_{:,i}^{l+1}| \right) \tag{7}$$

In this method, there is an *a priori* knowledge of the intended structure of the layer. We find that scaling the initial weights of each node by the inverse of its penalty (such that less relevant nodes have smaller weights from the outset) speeds up convergence and leads to more structured models. Therefore, after standard random initialization, we apply a $1/p[i]$ scaling to the weights of each node to promote a better separation of node importance from the outset.

In this technique, α can be thought of as establishing the relative importance of neurons, as it controls the growth rate of the penalization vector, while β controls the total strength of the regularization.

3 Experiments

The first set of experiments evaluates whether the proposed layers actually retain the most important information in the neurons at the front. We train a full-size network, then prune it by progressively removing higher-index nodes from the hidden layers, and measure how prediction quality is affected. We evaluate our method on the MNIST, Fashion-MNIST, CIFAR-10, Covertype, and Sensorless Drive Diagnosis (SDD) datasets. Results are compared against Random Pruning, Post-train Magnitude Pruning and a re-implementation of Group Lasso Pruning as described in [8]. These models are implemented with PyTorch[9]. The model pruning was performed using Torch Pruning[5]. The code can be found on the paper's repository.

Architectures, training epochs, batch sizes, number of batches used for readjustment, and experiment repetitions are listed in Table 1. Hyperparameters for Group Lasso Pruning and the Relevance-Sorted Network are described in Table 2. Training uses the Adam optimizer with a starting learning rate of $1e^{-3}$ and ReLU activations. Results are averaged accross multiple independent runs, repeated sufficiently to yield reliable confidence intervals (see Table 1).

Dataset	Architecture	Epochs	Adjustment Batches	Batch Size	Experiments
MNIST	784/400/300/100/10	5	40	64	10
Fashion-MNIST	784/400/300/100/10	5	40	64	10
CIFAR-10	3072/1024/512/256/10	10	50	64	10
Covertype	54/64/64/32/7	5	40	512	40
SDD	48/64/64/32/11	5	40	64	80

Table 1: Model architecture, epochs, adjustment batches, batch size and number of repetitions per dataset.

Dataset	Group Lasso	Relevance-Sorted Network
MNIST	$\alpha = 1e^{-3}$, $\beta = 1e^{-3}$	$\alpha = 0.5$, $\beta = 8e^{-2}$
Fashion-MNIST	$\alpha = 1e^{-3}$, $\beta = 1e^{-3}$	$\alpha = 0.5$, $\beta = 8e^{-2}$
CIFAR-10	$\alpha = 1e^{-4}$, $\beta = 1e^{-4}$	$\alpha = 0.5$, $\beta = 8e^{-2}$
Covertype	$\alpha = 4e^{-2}$, $\beta = 4e^{-2}$	$\alpha = 0.9$, $\beta = 8e^{-2}$
SDD	$\alpha = 4e^{-4}$, $\beta = 4e^{-4}$	$\alpha = 0.9$, $\beta = 5e^{-3}$

Table 2: Group Lasso Pruning and Relevance-Sorted Network hyper-parameters

In [7] and [8], Group Lasso Pruning is described as global pruning: the ranking of the nodes is computed over the whole network, so different layers may lose different fractions of nodes as long as the overall pruning percentage is met. This tends to work poorly with extreme levels of compression, where we observed entire layers being removed (which renders the network useless). Therefore, if more than 90% of the hidden nodes were to be removed (about 99% of the weights), we switched to local pruning, which ranks nodes only within their layer. The same applies to Post-train Magnitude Pruning.

Figure 1 shows the results of the experiment. A clear trend is that the models that perform worst under high compression are those with the best full-size performance. When tuning parameters for Group Lasso Pruning and Relevance-Sorted Networks, we observed that reducing the regularization strength shifts performance: it worsens results at high

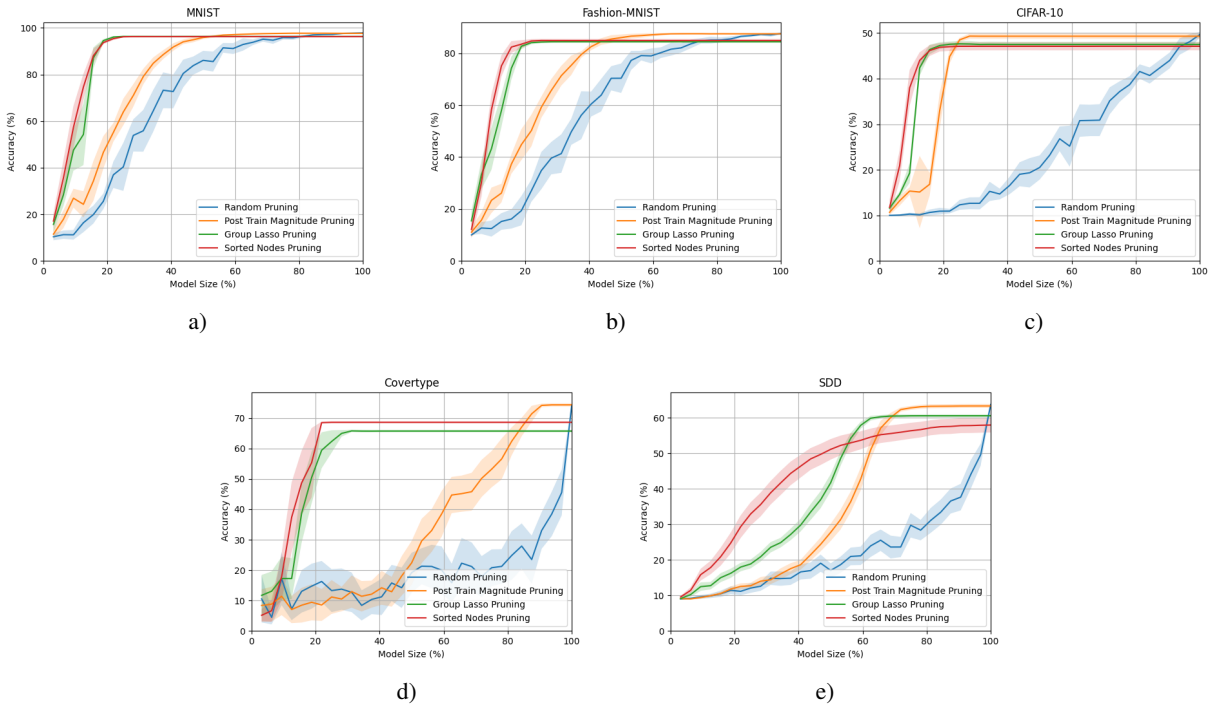


Figure 1: The first set of experiments trains a full-sized network and progressively removes neurons to obtain a pruned model. The experiments were run on a) MNIST, b) Fashion-MNIST, c) CIFAR-10, d) Covertype, and e) SDD. The solid lines are the mean observation and the shadowed areas represent the 95% confidence interval for the mean, assuming normal distribution. We define model size as the percentage of neurons remaining in the hidden layers, which is different from the percentage of *weights* remaining, a measure commonly used in the unstructured pruning literature.

compression while improving them at low compression. This indicates a trade-off between performance in high- and low-compression regimes.

With increasing compression, Random Pruning and Post-train Magnitude Pruning degrade fastest. On MNIST, Fashion-MNIST and CIFAR-10, Relevance-Sorted Network Pruning and Group Lasso Pruning perform similarly, although relevance-based pruning achieves slightly better results at high compression. On Covertype, Relevance-Sorted Networks outperform Group Lasso on the whole pruning spectrum. For SDD, both methods show similar full-size accuracy. Group Lasso Pruning behaves like Post-Train Magnitude pruning, with a plateau at the beginning but rapidly declining. The Relevance-Sorted Network has a more uniform decrease in performance and achieves much better results when the model size is less than 60%.

The second experiment focuses on the question of whether weight readjustment is needed after pruning. It proceeds exactly as the first, but trains the network on a small number of batches after pruning. The results are shown in Figure 2. The relative performance of the models remains unchanged. The models that performed worst in the first experiment benefit the most from the additional weight adjustment. Group Lasso Pruning and Relevance-Sorted Network Pruning also benefit from it in most cases, especially at high-compression levels. For instance, with Relevance-Sorted Networks and the Fashion-MNIST dataset, a network at 9.38% of its full size improves from 58.26% accuracy without adjustment to 82.99% with adjustment. Similar results can be observed in the other datasets. In Covertype, however, Relevance-Sorted Networks fail to improve as much as the other models do, especially Group Lasso.

4 Discussion

Our experiments demonstrate that the proposed Relevance-Sorted Networks enable the construction of neural networks in which nodes within each layer are ordered from most important to least important. This ordering can be used for network pruning by removing the least significant nodes.

Networks trained with Relevance-Sorted Nodes achieve pruning performance comparable to Group Lasso on the MNIST, Fashion-MNIST, and CIFAR-10 datasets. On the tabular datasets, Covertype and SDD, the results are less uniform, with

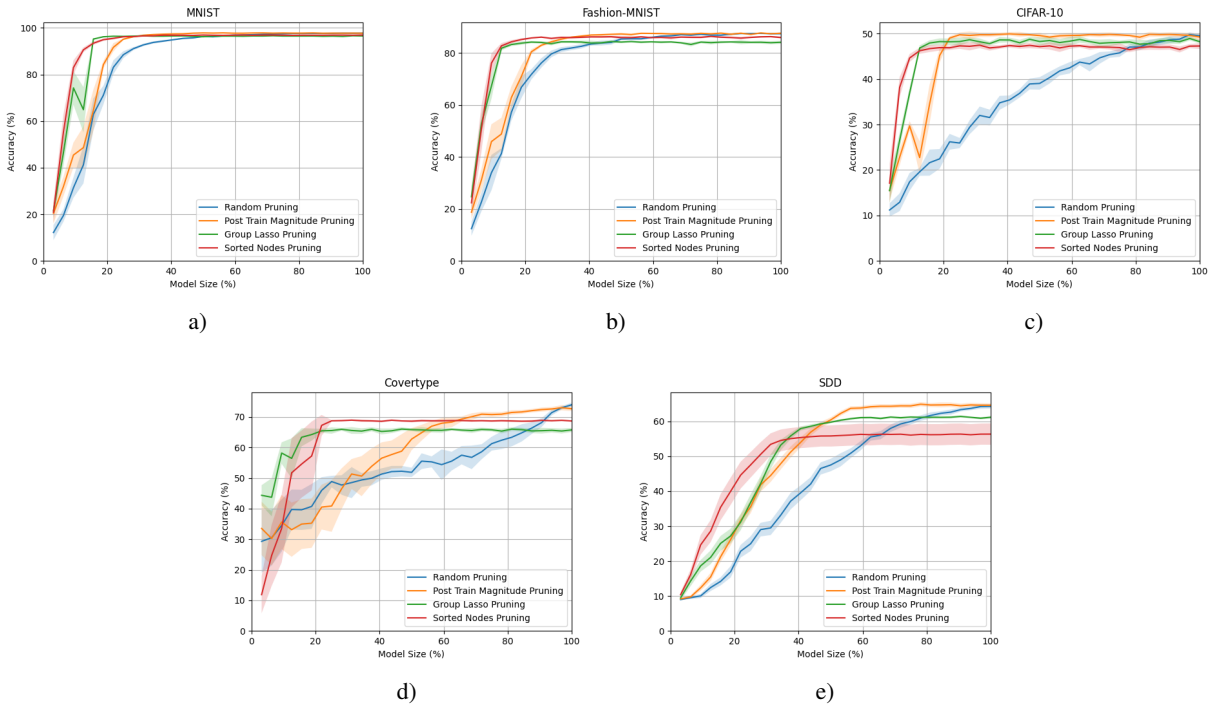


Figure 2: The second set of experiments trains a full-size network and removes neurons to obtain a pruned model, followed by a short training phase to adjust the weights to the new architecture. The experiments were run on a) MNIST, b) Fashion-MNIST, c) CIFAR-10, d) Covertype, and e) SDD. The strong lines are the mean observation and the shadowed areas represent the 95% confidence interval for the mean, assuming normal distribution. We define model size as the percentage of neurons remaining in the hidden layers, which is different from the percentage of *weights* remaining, a measure commonly used in the unstructured pruning literature.

relative performance depending on the dataset, the degree of compression, and whether a weight readjustment step is applied. We observe that Relevance-Sorted Networks have a visible advantage at high compression rates (below 10%).

Conceptually, both approaches are similar. They encourage nodes associated with weights of lower magnitude and regard such nodes as less important. Relevance-Sorted Nodes, however, add a notion of spatial ordering. Knowing node importance *a priori* allows this ordering to be encouraged from network initialization, which is not possible in Group Lasso Pruning.

From an implementation perspective, Relevance-Sorted Networks are slightly more complex to set up, as they require both *a priori* weight adjustment and an additional regularization term. However, the pruning process itself is simpler: removing contiguous rows or columns in the weight matrix rather than arbitrary selections in Group Lasso. Structured pruning libraries like Torch-Pruning succeed in abstracting this procedure and make both equally easy. Training-time computational overhead is minimal for both techniques.

5 Conclusion

The proposed method successfully orders nodes within each hidden layer by importance, enabling structured pruning of multilayer perceptrons without significant performance losses, which can result in energy and memory savings. Relevance-Sorted Network Pruning proves to be competitive with the other techniques studied, particularly Group Lasso Pruning, across all datasets studied. Our method achieves an overall better performance at high compression ratios, with the exception of the Covertype dataset when the network's weights are fine-tuned after pruning. Both techniques are similar in terms of training overhead and implementation complexity.

A direction for future research considers the expansion of this method to other types of neural networks, such as Convolutional Neural Networks and Transformers. This expansion is not trivial, since those networks have more complex structures and may require more elaborate solutions.

Acknowledgments

This work was made possible by a grant from the Calouste Gulbenkian Foundation. We thank the Foundation and the Mathematics committee of the Gulbenkian "Novos Talentos" scholarship program, Hugo Tavares and Jorge Freitas.

References

[1] Jaime Sevilla, Lennart Heim, Anson Ho, Tamay Besiroglu, Marius Hobbhahn, and Pablo Villalobos. Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, page 1–8. IEEE, July 2022.

[2] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10558–10578, 2024.

[3] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. In D. Touretzky, editor, *Advances in Neural Information Processing Systems*, volume 2. Morgan-Kaufmann, 1989.

[4] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2:129–146, 2020.

[5] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16091–16101, 2023.

[6] Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.

[7] Simone Scardapane, Danilo Comminiello, Amir Hussain, and Aurelio Uncini. Group sparse regularization for deep neural networks. *Neurocomputing*, 241:81–89, 2017.

[8] Tsubasa Ochiai, Shigeki Matsuda, Hideyuki Watanabe, and Shigeru Katagiri. Automatic node selection for deep neural networks using group lasso regularization. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5485–5489, 2017.

[9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA, 2019.